

Universidad Mayor de San Andrés
Facultad de Ciencias Puras y Naturales
Carrera de Informática



INFORME PENTESTING

Semestre I/2026

Estudiante: Riveros Vera Angeles Sarai

Materia: Taller Técnico Superior

Docente: Lic. Brigida Alexandra Carvajal Blanco

Fecha: 30 de Mayo de 2026

Fase explotación y Despliegue Local Ejecutada

Versión 2.0

La Paz - Bolivia

1. Resumen ejecutivo

El presente informe describe el proceso de evaluación de seguridad realizado sobre el sistema web “Pet Spa Grooming”, desarrollado por el estudiante Ivan Omar Limachi.

La evaluación tuvo como objetivo verificar el cumplimiento de controles básicos de seguridad relacionados con la validación de entradas, integridad en la lógica de negocio, autenticación, gestión de sesiones y control de acceso.

La metodología de revisión fue mixta: la sección inicial de autenticación y flujos administrativos fue analizada mediante una demostración en video proporcionada por el desarrollador (sin acceso directo al código fuente), mientras que el módulo transaccional de la tienda y el carrito de compras fue sometido a pruebas técnicas directas en un entorno local controlando e interceptando los parámetros del lado del cliente.

Durante la evaluación se evidenció la implementación de mecanismos de seguridad positivos como el hashing seguro de contraseñas, auditoría de acciones en el sistema y bloqueo temporal de cuentas tras intentos fallidos. Sin embargo, se identificó una falla crítica en la validación de datos del lado del servidor dentro del módulo de compras que compromete la integridad de la lógica de negocio y el control de inventarios, además de oportunidades de mejora en la robustez de los accesos.

2. Alcance de la prueba

Elemento	Descripción
Sistema objetivo	Sistema Web Spa de Mascotas
Tipo de sistema	Aplicación web Transaccional
Tecnologías identificadas	PHP, Apache, MySQL, HTML5 / JavaScript
Módulos evaluados	Login, registro de usuarios, auditoria, carrito de compras y confirmación de pedidos
Funcionalidades revisadas	Activación por correo, bloqueo de cuentas, hashing de contraseñas, auditoría
Periodo de evaluación	12/05/26 al 31/06/26
Responsable del sistema	Nina Limachi Ivan Omar
Evaluator	Riveros Vera Angeles Sarai
Evidencia analizada	https://www.youtube.com/watch?v=C_o

	zaWGDvCQ https://github.com/ivannn112233/spa?authuser=0
--	---

3. Metodología utilizada

La evaluación se ejecutó bajo un enfoque híbrido:

1. Análisis Pasivo: Observación funcional del comportamiento del sistema y configuraciones lógicas mediante la grabación de video para los módulos de administración y registro.
2. Análisis Activo (Pruebas de Caja Negra/Gris): Manipulación de parámetros en tiempo de ejecución (*Parameter Tampering*) utilizando las herramientas de desarrollo del navegador (*Chrome DevTools*), analizando de forma directa las peticiones HTTP POST, cargas útiles (*Payloads*) y las respuestas de redirección (**302 Found**) del servidor.

3.1. Recolección de información

Durante la demostración se identificaron los siguientes controles de seguridad:

Control observado	Estado	Descripción técnica
Registro libre de clientes	Implementado	Acceso abierto para creación de cuentas de usuarios base.
Registro manual de empleados	Implementado	Restringido exclusivamente al rol de Administrador.
Activación por correo electrónico	Implementado	Envío de enlaces automatizado para habilitar cuentas.
Seguridad de contraseñas	Implementado	Se muestran instrucciones normativas de complejidad y una validación dinámica en tiempo real.
Medidor de fuerza - Contraseñas	Implementado	
Email de Activación	Implementado	Almacenamiento no en texto plano dentro de la base de datos.

Expiración de enlaces de activación	No evidenciado	
Bloqueo por intentos fallidos	Implementado	Cierre temporal de la cuenta por 15 minutos tras superar el límite de intentos.
Hashing seguro de contraseñas	Implementado	Almacenamiento no en texto plano dentro de la base de datos
Auditoría	Implementado	Bitácora interna que registra las acciones críticas ejecutadas por los usuarios.
Borrado lógico	No implementado	
Autenticación 2FA	No implementado	Ausencia de un segundo factor para roles críticos.
Inicio de sesión/continuar con Google	No implementado	

3.2. Observaciones adicionales

- El sistema genera contraseñas temporales automáticas para nuevos empleados. No obstante, no obliga a cambiarlas de manera forzosa en el primer inicio de sesión, dejándolo como una acción opcional.

4. Hallazgos encontrados

Código	Observación	Módulo	Riesgo	Descripción	Recomendación
V-01	Ausencia de autenticación 2FA	Login	Medio	No se evidenció autenticación multifactor durante la demostración	Implementar autenticación multifactor
V-02	Inicio de sesión con Google no implementado	Login	Medio	El sistema no cuenta con autenticación mediante proveedores externos	Considerar integración futura con Google OAuth
V-03	Borrado lógico no	Gestión	Alto	No se evidenció	Realizar una

	implementado	de datos		mecanismo de eliminación lógica de registros	demostración más profunda para corroborar este punto.
V-04	Manipulación de parámetros e inyecciones de Cantidades Negativas	Tienda/carrito	Alto	El script <code>actualizar_carrito.php</code> acepta enteros negativos (ej. -2). El backend almacena estos datos en la sesión, alterando los subtotales algebraicamente e impactando de forma directa en las consultas SQL de inventario (<code>\text{Stock} - (-2) = \text{Stock} + 2</code>). <code>if (\$cantidad <= 0) { throw Error; }</code>	Implementar una validación estricta en el servidor que rechace peticiones si la cantidad recibida es ≤ 0\$:
V-05	Validación limitada por Restricción de Entorno	General	Bajo	Ciertas capas de código de la sección administrativa (configuraciones internas) solo pudieron auditarse vía video, reduciendo la exhaustividad del testeo	

Las vulnerabilidades encontradas fueron clasificadas según su impacto.

Nivel	Descripción
Alto	Vulnerabilidades que afectan la seguridad crítica
Medio	Riesgos explotables bajo ciertas condiciones
Bajo	Riesgo de seguridad baja

5. Evidencias

Explotación del Flujo de Compra (Parameter Tampering)

Las evidencias obtenidas corresponden únicamente a capturas extraídas de la demostración en video proporcionada por el estudiante que desarrolló el sistema.

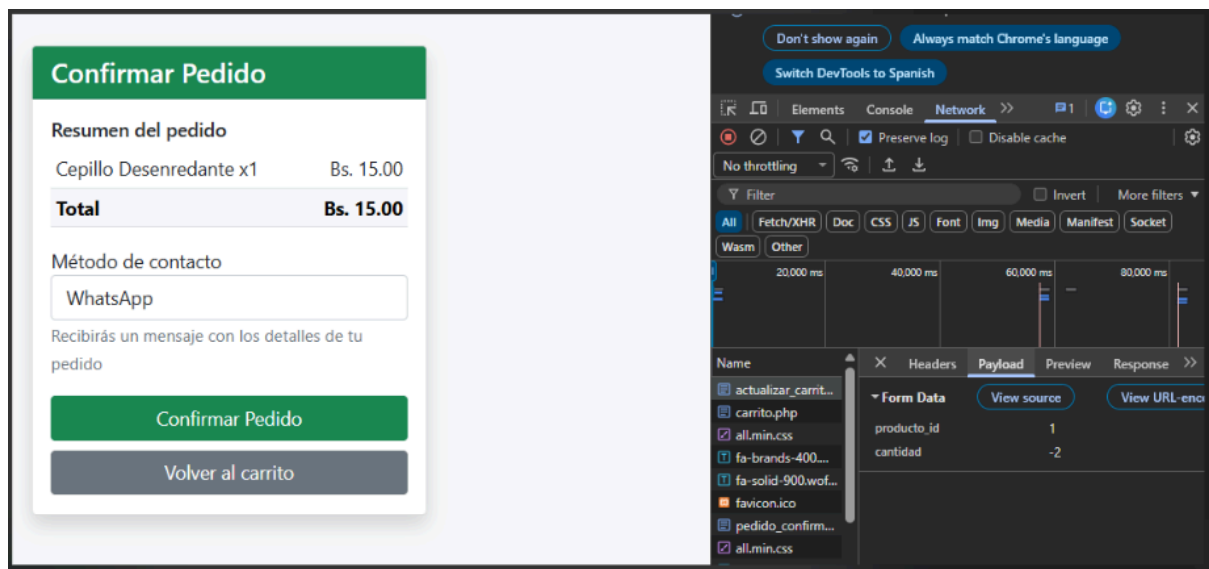
A. Manipulación de Petición HTTP POST:

Al alterar las restricciones del lado del cliente (`min="1"`) e inyectar un valor negativo de `-2` para el *Shampoo Hipoalergénico* (`producto_id: 1`), el navegador envía con éxito la carga útil anómala a través del método POST:

Payload Capturado:

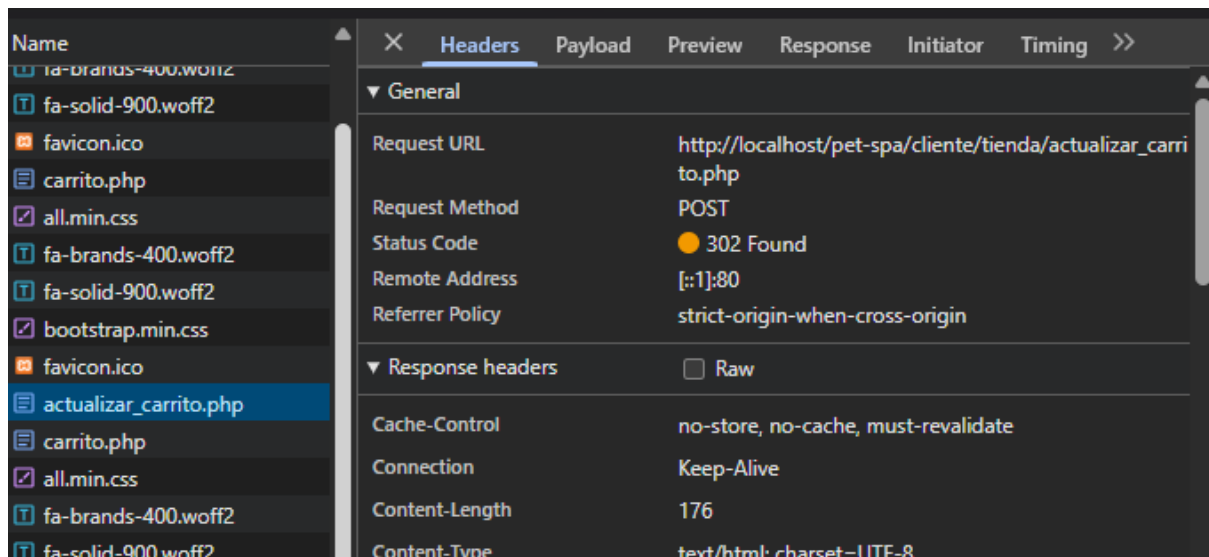
`producto_id: 1`

`cantidad: -2`



B. Procesamiento Exitoso del Servidor (Redirección 302):

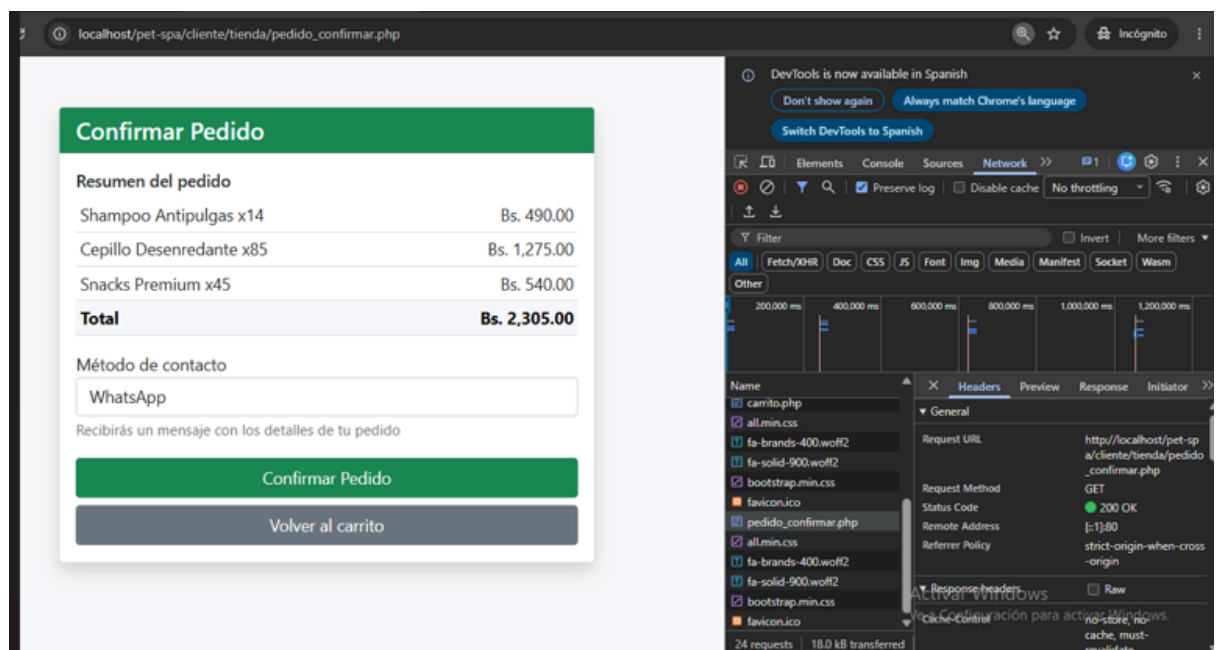
El archivo `actualizar_carrito.php` procesa la cantidad negativa sin oponer resistencia, respondiendo con un código de estado `302 Found`, validando indirectamente la transacción e internalizando la matemática corrupta en la sesión del servidor.



C. Efecto de Persistencia en la Confirmación del Pedido

(**pedido_confirmar.php**):

Al avanzar al módulo final de facturación, la aplicación arrastra el impacto del cálculo. Aunque oculta visualmente el producto negativo para intentar limpiar la interfaz, el total final de la orden mantiene el valor modificado por el descuento matemático del elemento ocultado, demostrando que la lógica interna del negocio se ha quebrado y es vulnerable a fraudes transaccionales y de inventario.



Controles de Seguridad Consolidados

No todas las pruebas resultaron en vulnerabilidades. El sistema evidenció un correcto diseño de seguridad en los siguientes componentes evaluados:

1. Módulo de Autenticación (Inyección SQL)

Se confirma la resiliencia del inicio de sesión frente a ataques de inyección de código (SQLi) mediante el manejo correcto de excepciones de autenticación. Tras saltarse las validaciones estéticas de formato del frontend y enviar payloads maliciosos estructurados como `' OR '1'='1'`, el sistema devolvió un mensaje controlado de credenciales incorrectas, lo que comprueba el uso seguro de consultas preparadas parametrized en el backend.

2. Gestión de Intentos de Acceso (Fuerza Bruta)

El mecanismo de bloqueo temporal de cuentas mitiga con éxito los ataques de fuerza bruta automatizados, al persistir el estado de penalización directamente en la lógica del servidor y de manera independiente al navegador del cliente. Las pruebas cruzadas desde navegadores paralelos e incógnito demostraron que el bloqueo persiste a nivel de base de datos y rastrea el tiempo exacto remanente para la liberación del usuario.

3. Control de Inyección de Código (Stored XSS en Registro de Mascotas)

Se realizaron pruebas de estrés inyectando los vectores `<script>alert(1)</script>` y `<img src=x onerror=alert(1)>` en el formulario de mascotas. El sistema demostró ser seguro debido a que implementa un filtrado de sanitización de salida eficiente (codificación de entidades HTML). Las etiquetas se imprimen de forma literal como texto inofensivo dentro del contenedor `<h5 class="mascota-nombre">`, transformando los caracteres en variables seguras (`<` y `>`) y neutralizando cualquier ejecución arbitraria de JavaScript.

4. Control de Concurrencia e Integridad de Datos (Módulo de Citas)

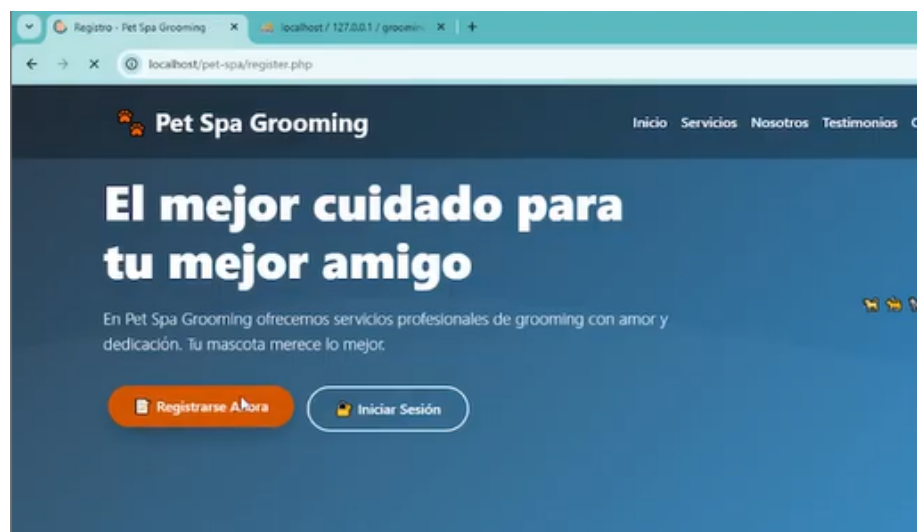
Se evaluó el sistema ante condiciones de carrera (*Race Conditions*) registrando múltiples citas en el mismo bloque horario desde distintos entornos. El backend mitigó el ataque de forma satisfactoria rechazando las solicitudes duplicadas, lo que comprueba la existencia de controles de validación de disponibilidad o índices únicos estrictos en la base de datos.

5. Gestión e Invalidation de Sesiones (*Session Termination*)

Se comprobó la consistencia en el ciclo de vida de los identificadores de sesión. Al iniciar sesión en múltiples pestañas y ejecutar la acción de salida en una de ellas, el servidor destruyó inmediatamente el estado de la sesión activa (`session_destroy()`). Al refrescar las ventanas paralelas, el sistema las expulsó de forma automática hacia el login, impidiendo accesos huérfanos.

Durante la evaluación se observaron las siguientes funcionalidades:

- Inicio de sesión.



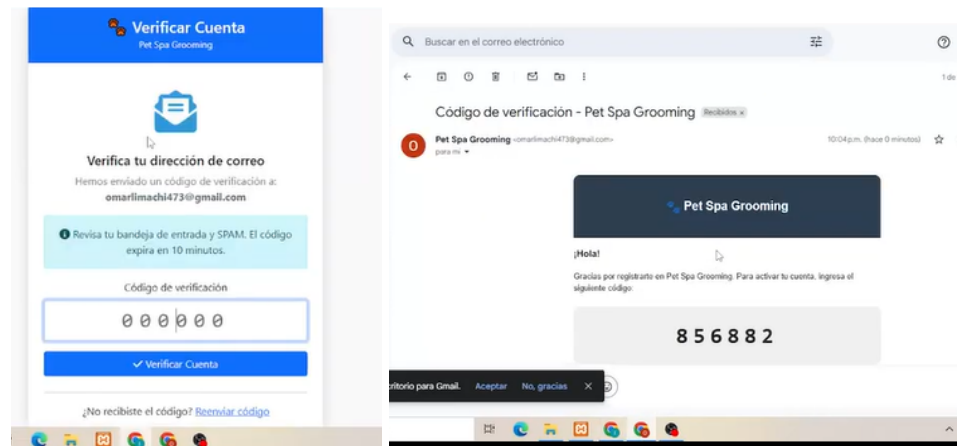
- Registro de usuarios.

Two side-by-side screenshots of web forms. The left screenshot is titled 'Registro de Cliente' and contains sections for 'Datos de acceso' (Email, Password, Confirm Password) and 'Datos personales' (Name, Surname, Phone, ID Card). The right screenshot is titled 'Crear Nuevo Usuario' and includes a note about a temporary password, fields for Email and Role, and 'Datos personales' (Name, Surname, Phone). It also has a checkbox for 'Enviar credenciales por email' and buttons for 'Crear Usuario' and 'Cancelar'.

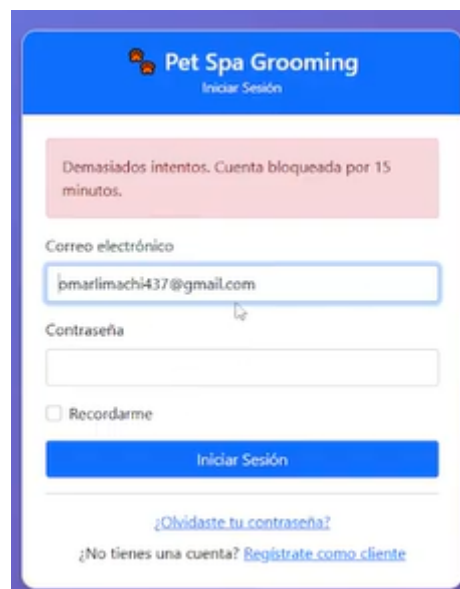
Registro libre cliente

Registro de empleados, solo admini

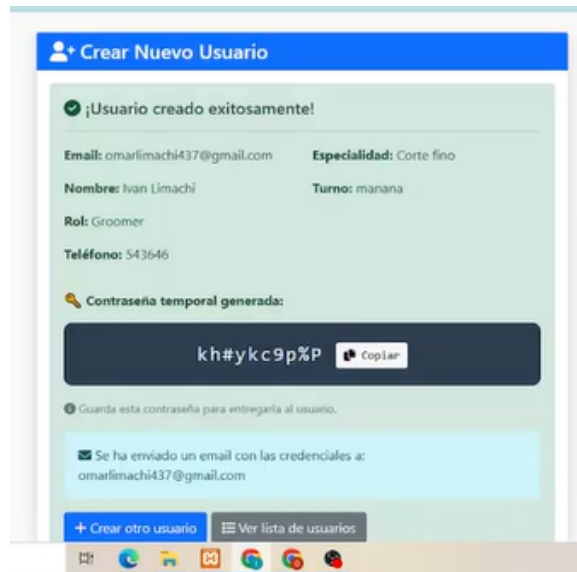
- Activación de cuentas mediante correo electrónico.



- Bloqueo temporal de cuentas tras múltiples intentos fallidos.



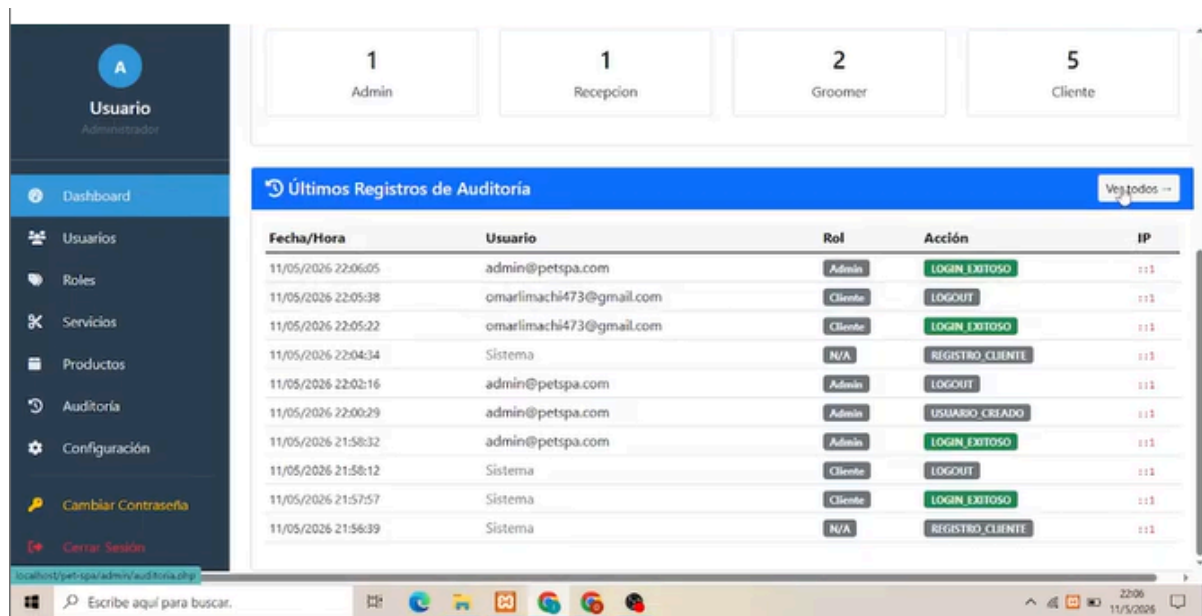
- Generación automática de contraseñas temporales para empleados.



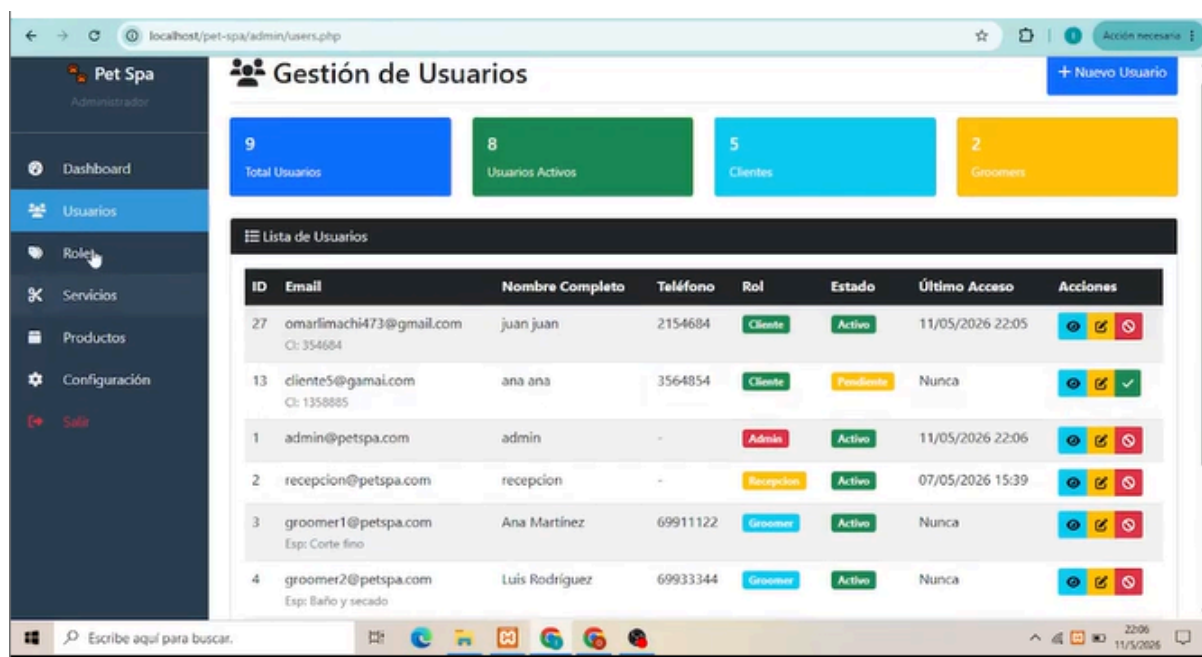
- Cambio obligatorio de contraseña en el primer inicio de sesión.



- Registro de auditoría.



- Gestión de usuarios.



6. Recomendaciones generales

- **Prioridad 1 (Urgente):** Sanitizar rigurosamente todas las entradas numéricas del lado del servidor en formularios comerciales. Las restricciones en HTML5/JavaScript son solo herramientas estéticas y deben considerarse inseguras por defecto.
- **Establecer un Flujo Forzoso de Contraseñas Temporales:** Modificar la lógica de acceso para empleados de modo que, si detecta una contraseña autogenerada por primera vez, el sistema redirija obligatoriamente a un formulario de cambio de clave antes de permitir la navegación por el panel.

- **Robustecer enlaces de activación:** Encriptar y añadir marcas de tiempo (*timestamps*) a las URL generadas para funciones de correo electrónico.
- **Implementar HTTPS obligatorio:** Asegurar que todo el tráfico local y de producción viaje cifrado para evitar la interceptación de cookies de sesión o payloads transaccionales.

7. Conclusiones

La evaluación realizada sobre el sistema web “**Pet Spa Grooming**” determinó que la aplicación posee un nivel de seguridad muy competente en sus pilares de autenticación, control de sesiones, protección contra XSS y consistencia de datos recurrentes.

No obstante, la confianza ciega en los parámetros numéricos de entrada dentro del carrito de compras genera una ventana de vulnerabilidad crítica que empaña la robustez general de la plataforma. La aplicación inmediata del parche propuesto del lado del servidor para restringir valores menores o iguales a cero asegurará el entorno transaccional, convirtiendo al sistema en un desarrollo altamente seguro y listo para producción.